

Welche Engine fuer professionell?

Von der Ursina-Lernversion zur echten PixelPortalRift-Engine

Ursina war perfekt, um 3D-Grundideen zu lernen: Entity, update(), Kamera, Collider. Fuer eine echte, professionelle Version von PixelPortalRift stoessst Ursina aber an Grenzen.

In diesem Heft:

1. Warum Ursina eine Lern-Engine ist, keine Produktions-Engine
2. Vergleich: Godot, Unity und Unreal
3. Warum Godot 4 die Empfehlung ist
4. Der Migrationspfad in Phasen
5. Erste Schritte und Links

1. Warum Ursina eine Lern-Engine ist

Ursina ist ein duenner, freundlicher Wrapper um die Engine Panda3D. Genau das macht sie so gut zum Lernen: wenige Zeilen, sofort ein Ergebnis. Fuer eine professionelle, veroeffentlichbare Voxel-Welt wie PixelPortalRift fehlen ihr aber wichtige Bausteine.

1.1 Die Grenzen im Detail

Thema	Ursina	Was 'professionell' braucht
Voxel-Performance	Jeder Block ist ein einzelnes Entity - bei grossen Welten bricht die Framerate ein	Chunk-Meshing: viele Bloecke werden zu einem Mesh zusammengefasst
Ladeweiten	Keine eingebaute Chunk-Verwaltung, Welt muss man selbst im Speicher halten	Streaming: nur die Umgebung des Spielers wird geladen
Tooling	Kein visueller Editor, alles ist Code	Editor mit Szenen-Baum, Debugger, Profiler
Export/Plattformen	Vor allem PC, wenig fertige Export-Pipelines	Fertige Exporte fuer Windows/Mac/Linux, teils Mobile/Web
Community/Assets	Klein, wenig Beispiele fuer grosse Projekte	Grosse Community, fertige Voxel-Plugins verfuegbar

Kurz gesagt: Ursina ist bewusst einfach gehalten, damit man schnell lernt. Ein Spiel, das viele Spieler laden und dauerhaft spielen sollen, braucht eine Engine mit mehr Werkzeugen darum herum.

2. Vergleich der Kandidaten

2.1 Godot 4

Kostenlos, Open Source, eigener Editor. Skriptsprache GDScript ist leicht lesbar (aehnlich Python), C# geht auch. Es gibt ein fertiges Community-Modul fuer Voxel-Welten (godot_voxel von Zylann), das genau fuer Minecraft-artige Bloeckewelten gebaut ist: Chunk-Streaming, automatisches Meshing, Level-of-Detail.

2.2 Unity

Sehr grosses Ökosystem und viele fertige Voxel-Assets im Asset Store. Skriptsprache ist C#. Editor und Build-Pipeline sind maechtig, aber auch schwerer als bei Godot. Lizenzbedingungen sollte man vor einem kommerziellen Projekt genau pruefen.

2.3 Unreal Engine

Technisch am staerksten (Grafik, Physik, Netzwerk), aber fuer ein Schueler- bzw. Solo-Projekt meist ueberdimensioniert: C++ oder Blueprints, hohe Lernkurve, sehr grosse Projektdateien und Builds.

2.4 Vergleichstabelle

Kriterium	Godot 4	Unity	Unreal
Kosten	Kostenlos	Kostenlos bis Umsatzgrenze	Kostenlos bis Umsatzgrenze
Sprache	GDScript / C#	C#	C++ / Blueprints
Lernkurve	Niedrig-mittel	Mittel	Hoch
Voxel-Tools	godot_voxel Plugin	Asset-Store-Plugins	Wenige, meist selbst gebaut
Projektgroesse	Klein/leichtgewichtig	Mittel-gross	Gross

3. Empfehlung: Godot 4

Fuer die professionelle Version von PixelPortalRift ist Godot 4 die sinnvollste Wahl.

3.1 Gruende

- Godot wird bei einem anderen Projekt (Monster-RPG) bereits produktiv genutzt - die GDScript-Erfahrung ist direkt uebertragbar.
- Kostenlos und Open Source, keine Ueberraschungen bei Lizenz oder Kosten.
- godot_voxel-Plugin loest genau die Probleme, an denen Ursina bei grossen Welten scheitert (Chunk-Meshing, Streaming, LOD).
- Ausgereifter Editor mit Szenen-Baum, Debugger und fertigen Exportern fuer Windows/Mac/Linux.
- Die Konzepte aus den Ursina-Uebungen (Entity/Node-Denken, update()-Schleife, 3D-Koordinaten x/y/z) uebertragen sich fast eins zu eins auf Godot-Nodes.

3.2 Begriffe: Ursina zu Godot

Ursina-Begriff	Godot-Entsprechung
Entity	Node3D (bzw. spezialisierter Node, z.B. MeshInstance3D)
app.run()	Der Godot-Editor startet die Szene, eigene Main-Loop entfaellt
update()	_process(delta) oder _physics_process(delta)
held_keys["d"]	Input.is_action_pressed(...) ueber die Input-Map
collider="box"	CollisionShape3D mit BoxShape3D
FirstPersonController	CharacterBody3D + eigenes oder Community-Skript

4. Migrationspfad in Phasen

Ein direkter Sprung von der Pygame-Logik auf eine Produktions-Engine ist ein grosser Schritt. Sinnvoll ist ein Zwischenschritt ueber Ursina als Lernstufe, danach der Umstieg auf Godot fuer die echte Version.

Phase	Ziel	Engine
Phase 1 (erledigt)	3D-Grundideen lernen: Entity, update(), Kamera, Collider	Ursina
Phase 2	Kleine Godot-Testszene: Wuerfel, Boden, Bewegung, Kollision	Godot 4
Phase 3	godot_voxel einbinden, erste Chunk-Welt generieren	Godot 4 + godot_voxel
Phase 4	Bestehende Spiellogik uebertragen: Inventar, Crafting, Portale, Mobs	Godot 4
Phase 5	Politur, Speichersystem, Export fuer Windows/Mac/Linux	Godot 4

4.1 Was aus PixelPortalRift direkt uebernommen werden kann

- Die Datenstrukturen aus constants.py (BLOCK_PROPERTIES, ITEM_PROPERTIES, DIMENSIONS) lassen sich fast unveraendert als GDScript-Dictionaries oder Godot-Resources uebernehmen.
- Die Spiellogik (Crafting-Regeln, Portal-Bedingungen, Mob-Verhalten) ist Engine-unabhaengig und kann Schritt fuer Schritt uebersetzt werden.
- Die Erfahrung mit Voxel-Culling aus der Ursina-Phase (nur sichtbare Blockflaechen rendern) hilft direkt beim Verstehen, was godot_voxel automatisch fuer euch erledigt.

5. Erste Schritte

5.1 Godot installieren und testen

```
# Godot 4 von godotengine.org herunterladen (keine Installation noetig,
# einzelne .exe/AppImage-Datei)
# Neues 3D-Projekt anlegen, dann als ersten Test:

extends Node3D

func _ready():
    var cube = MeshInstance3D.new()
    cube.mesh = BoxMesh.new()
    add_child(cube)
```

5.2 godot_voxel einbinden

- Plugin von github.com/Zylann/godot_voxel in den Projektordner unter addons/ legen.
- In den Projekteinstellungen unter 'Plugins' aktivieren.
- Mit einer VoxelTerrain-Node und einem einfachen Generator-Script starten, bevor eigene Bloecke definiert werden.

5.3 Tipps fuer den Umstieg

- Nicht alles auf einmal umbauen: zuerst eine winzige Godot-Testszene, dann Stueck fuer Stueck PixelPortalRift-Funktionen nachbauen.
- Die alte Pygame- und die Ursina-Version bleiben als Referenz bestehen - nichts loeschen, solange die Godot-Version noch nicht laeuft.
- Bei godot_voxel-Fragen zuerst die Projekt-Doku und Beispiele im Repository pruefen, dort gibt es fertige Beispielszenen.

Wo nachschlagen

- Godot-Dokumentation: docs.godotengine.org
- godot_voxel-Plugin und Beispiele: github.com/Zylann/godot_voxel
- Bereits vorhandenes Godot-Wissen aus dem eigenen Monster-RPG-Projekt laesst sich direkt wiederverwenden.

Zusammengefasst: Ursina hat seinen Zweck erfuehlt - ihr versteht jetzt, wie 3D-Engines ticken. Fuer die professionelle Version von PixelPortalRift ist Godot 4 mit dem godot_voxel-Plugin der naechste, sinnvolle Schritt.